

CSAI 204 – Operating Systems

Lab 4 Report

System Calls & Pipe Communication in xv6

Introduction

This report explains the implementation of additional system calls inside the xv6 operating system along with a practical example of interprocess communication using pipes in C. The work focused on modifying kernel components, creating user-level test programs, and reproducing a Unix-style pipeline.

Background

In xv6, introducing a new system call requires coordinated modifications across several kernel files. These updates connect the user-space function to the kernel handler responsible for executing the call. The exercise also demonstrates how pipes can transfer data between multiple processes.

Part 1: Creating the hello() System Call

The hello() system call was implemented to display a greeting message when invoked from a user program.

Kernel Integration

A unique syscall identifier was assigned in syscall.h. The prototype and dispatch table entries were then registered in syscall.c to connect the handler with the kernel dispatcher.

Handler Implementation

Inside sysproc.c, the handler was written to print a simple message using the xv6 console output functions.

User-Space Access

The usys.pl script and user.h header file were updated so user programs could invoke the system call directly.

Testing

A small test program was compiled and added to the Makefile, allowing the new syscall to be executed from the xv6 shell.

Part 2: Implementing helloYou(name)

The second system call expanded functionality by accepting a string argument from user space.

System Call Registration

Similar modifications were applied across the kernel source files to register the new syscall number and link it with its implementation.

Passing Arguments

The kernel handler retrieved the user-provided string using xv6 argument handling utilities before displaying a personalized greeting message.

User Program

A dedicated test application was created to pass a sample name into the syscall and verify correct execution.

Part 3: Recreating ls | grep "txt" Using Pipes

This section recreated the behavior of a Unix shell pipeline using low-level system calls in C.

Pipe Communication

The program created a communication channel using `pipe()`, then generated child processes using `fork()`.

Input and Output Redirection

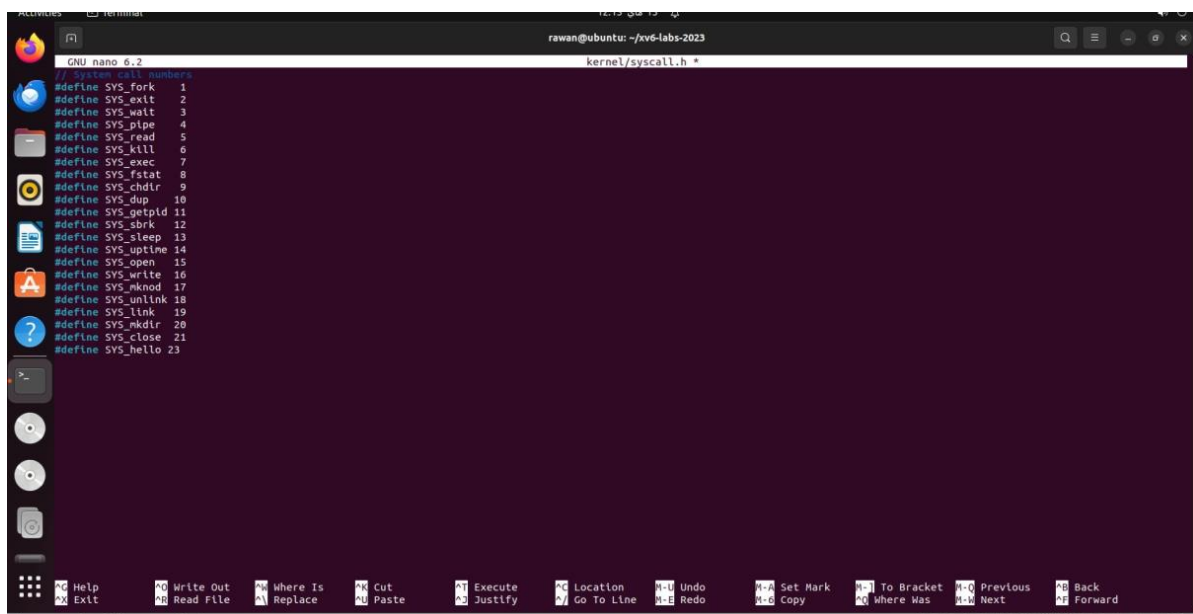
The first child redirected standard output toward the pipe before executing `ls`, while the second child redirected standard input from the pipe and executed `grep` with the pattern "txt".

Execution Results

After compilation, the program successfully filtered directory contents and displayed only files containing the `txt` extension, matching the behavior of the original shell command.

Screenshots

The following figures show the modified kernel files, test programs, Makefile updates, and terminal outputs generated during execution.



```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 kernel/syscall.c
argaddr(int n, uint64 *lp)
{
    *lp = argraw(n);
}

// Fetch the nth word-sized system call argument as a null-terminated string.
// Copies into buf, at most max.
// Returns string length if OK (including null), -1 if error.
int
argstr(int n, char *buf, int max)
{
    uint64 addr;
    argaddr(n, &addr);
    return fetchstr(addr, buf, max);
}

// Prototypes for the functions that handle system calls.
extern uint64 sys_hello(void);
extern uint64 sys_exit(void);
extern uint64 sys_wait(void);
extern uint64 sys_pipe(void);
extern uint64 sys_read(void);
extern uint64 sys_kill(void);
extern uint64 sys_exec(void);
extern uint64 sys_fstat(void);
extern uint64 sys_chdir(void);
extern uint64 sys_dup(void);
extern uint64 sys_getpid(void);
extern uint64 sys_sbrk(void);
extern uint64 sys_sleep(void);
extern uint64 sys_uptime(void);
extern uint64 sys_open(void);
extern uint64 sys_write(void);
extern uint64 sys_mknod(void);
extern uint64 sys_unlink(void);
extern uint64 sys_link(void);
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was Previous Next Back Forward
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 kernel/syscall.c
extern uint64 sys_getpid(void);
extern uint64 sys_sbrk(void);
extern uint64 sys_sleep(void);
extern uint64 sys_uptime(void);
extern uint64 sys_open(void);
extern uint64 sys_write(void);
extern uint64 sys_mknod(void);
extern uint64 sys_unlink(void);
extern uint64 sys_link(void);
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
    [SYS_fork]    sys_fork,
    [SYS_exit]    sys_exit,
    [SYS_wait]    sys_wait,
    [SYS_pipe]    sys_pipe,
    [SYS_read]    sys_read,
    [SYS_kill]    sys_kill,
    [SYS_exec]    sys_exec,
    [SYS_fstat]   sys_fstat,
    [SYS_chdir]   sys_chdir,
    [SYS_dup]     sys_dup,
    [SYS_getpid]  sys_getpid,
    [SYS_sbrk]    sys_sbrk,
    [SYS_sleep]   sys_sleep,
    [SYS_uptime]  sys_uptime,
    [SYS_open]    sys_open,
    [SYS_write]   sys_write,
    [SYS_mknod]   sys_mknod,
    [SYS_unlink]  sys_unlink,
    [SYS_link]    sys_link,
    [SYS_mkdir]   sys_mkdir,
    [SYS_close]   sys_close,
    [SYS_hello]   sys_hello,
};

void
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 kernel/sysproc.c
acquire(&tickslock);
ticks0 = ticks;
while(ticks - ticks0 < n){
    if(killed(myproc())){
        release(&tickslock);
        return -1;
    }
    sleep(&ticks, &tickslock);
}
release(&tickslock);
return 0;
}

uint64
sys_kill(void)
{
    int pid;

    argint(0, &pid);
    return kill(pid);
}

// return how many clock tick interrupts have occurred
// since start.
uint64
sys_uptime(void)
{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

uint64
sys_hello(void)
{
    printf("Hello\n");
    return 0;
}
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 user/usys.pl *
#!/usr/bin/perl -w

# Generate usys.S, the stubs for syscalls.

print "# generated by usys.pl - do not edit\n";
print "#include \"kernel/syscall.h\"\n";

sub entry {
    my $name = shift;
    print "global $name\n";
    print "$name:\n";
    print "    ll a7, Sys_S($name)\n";
    print "    reall\n";
    print "    ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("sleep");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("fork");
entry("sleep");
entry("uptime");
entry("hello");
```

```
GNU nano 6.2 user/user.h *
struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(const char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int hello(void);

// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void* memmove(void*, const void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
void fprintf(int, const char*, ...);
void printf(const char*, ...);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
void* malloc(uint);
void free(void*);
int atoi(const char*);
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 user/testhello.c *
#include "kernel/types.h"
#include "user/user.h"

int
main()
{
    hello();
    exit(0);
}
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2                                     Makefile *
```

```
$U/usys.o : $U/usys.S
$(CC) $(CFLAGS) -c -o $U/usys.o $U/usys.S

$U/_forktest: $U/forktest.o $(ULIB)
# forktest has less library code linked in - needs to be small
# in order to be able to max out the proc table.
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/forktest.o $U/ulib.o $U/usys.o
$(OBJDUMP) -S $U/_forktest > $U/forktest.asm

mkfs/mkfs: mkfs/mkfs.c $K/fs.h $K/param.h
gcc $(XCFLAGS) -Werror -Wall -I. -o mkfs/mkfs mkfs/mkfs.c

# Prevent deletion of intermediate files, e.g. cat.o, after first build, so
# that disk image changes after first build are persistent until clean. More
# details:
# http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
.PRECIOUS: %.o

UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_pingpong\
    $U/_testhello\
```

```
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -fstack-protector -fno-pie -no-pie -c -o user/pingpong.o user/pingpong.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_pingpong
riscv64-linux-gnu-objdump -S user/_pingpong > user/pingpong.asm
riscv64-linux-gnu-objdump -t user/_pingpong | sed '1,/SYMBOL TABLE/d'
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -fstack-protector -fno-pie -no-pie -c -o user/testhello.o user/testhello.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_testhello
riscv64-linux-gnu-objdump -S user/_testhello > user/testhello.asm
riscv64-linux-gnu-objdump -t user/_testhello | sed '1,/SYMBOL TABLE/d'
mkfs/mkfs fs.img README user/xargstest.sh user/_cat user/_echo user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_pingpong user/_testhello
mkfs: meta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1)
mkfs: balloc: first 815 blocks have been allocated
mkfs: balloc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -s -S -nographic -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

part 2 starting
part 1 starting
init: starting sh
$ testhello
Hello
$
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 kernel/syscall.h *
// System call numbers
#define SYS_fork 1
#define SYS_exit 2
#define SYS_wait 3
#define SYS_pipe 4
#define SYS_read 5
#define SYS_kill 6
#define SYS_exec 7
#define SYS_fstat 8
#define SYS_chdir 9
#define SYS_dup 10
#define SYS_getpid 11
#define SYS_sbrk 12
#define SYS_sleep 13
#define SYS_uptime 14
#define SYS_open 15
#define SYS_write 16
#define SYS_mknod 17
#define SYS_unlink 18
#define SYS_link 19
#define SYS_mkdir 20
#define SYS_close 21
#define SYS_hello 23
#define SYS_helloYou 24
```

```
GNU nano 6.2 kernel/syscall.c *
{
    *lp = argraw(n);
}

// Fetch the nth word-sized system call argument as a null-terminated string.
// Copies into buf, at most max.
// Returns string length if OK (including nul), -1 if error.
int
argstr(int n, char *buf, int max)
{
    uint64 addr;
    argaddr(n, &addr);
    return fetchstr(addr, buf, max);
}

// Prototypes for the functions that handle system calls.
extern uint64 sys_fork(void);
extern uint64 sys_hello(void);
extern uint64 sys_helloYou(void);
extern uint64 sys_exit(void);
extern uint64 sys_wait(void);
extern uint64 sys_pipe(void);
extern uint64 sys_read(void);
extern uint64 sys_kill(void);
extern uint64 sys_exec(void);
extern uint64 sys_fstat(void);
extern uint64 sys_chdir(void);
extern uint64 sys_dup(void);
extern uint64 sys_getpid(void);
extern uint64 sys_sbrk(void);
extern uint64 sys_sleep(void);
extern uint64 sys_uptime(void);
extern uint64 sys_open(void);
extern uint64 sys_write(void);
extern uint64 sys_mknod(void);
extern uint64 sys_unlink(void);
extern uint64 sys_link(void);
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);
```

```
GNU nano 6.2 kernel/syscall.c *
[SYS_wait] sys_wait,
[SYS_pipe] sys_pipe,
[SYS_read] sys_read,
[SYS_kill] sys_kill,
[SYS_exec] sys_exec,
[SYS_fstat] sys_fstat,
[SYS_chdir] sys_chdir,
[SYS_dup] sys_dup,
[SYS_getpid] sys_getpid,
[SYS_sbrk] sys_sbrk,
[SYS_sleep] sys_sleep,
[SYS_uptime] sys_uptime,
[SYS_open] sys_open,
[SYS_write] sys_write,
[SYS_mknod] sys_mknod,
[SYS_unlink] sys_unlink,
[SYS_link] sys_link,
[SYS_mkdir] sys_mkdir,
[SYS_close] sys_close,
[SYS_hello] sys_hello,
[SYS_helloYou] sys_helloYou,
};
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 user/user.h *
struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(const char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int hello(void);
int helloYou(char *);

// ulib.c
int stat(const char*, struct stat*);
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 user/usys.pl *
#!/usr/bin/perl -w

# Generate usys.S, the stubs for syscalls.

print "# generated by usys.pl - do not edit\n";

print "#include \"kernel/syscall.h\"\n";

sub entry {
    my $name = shift;
    print ".global $name\n";
    print "${name}:\n";
    print "    li a7, SYS_${name}\n";
    print "    ecall\n";
    print "    ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("sleep");
entry("uptime");
entry("hello");
entry("helloYou");
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 kernel/sysproc.c *

uint64
sys_kill(void)
{
    int pid;

    argint(0, &pid);
    return kill(pid);
}

// return how many clock tick interrupts have occurred
// since start.
uint64
sys_uptime(void)
{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

uint64
sys_hello(void)
{
    printf("Hello\n");
    return 0;
}

uint64
sys_helloYou(void)
{
    char name[50];

    argstr(0, name, 50);
    printf("Hello %s\n", name);
    return 0;
}
```

```
ties Terminal 12:23 مای 15
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 user/testhelloyou.c *

#include "kernel/types.h"
#include "user/user.h"

int
main()
{
    helloYou("Rawan");
    exit(0);
}
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 Makefile *
SU/_forktest\
SU/_grep\
SU/_init\
SU/_kill\
SU/_ln\
SU/_ls\
SU/_mkdir\
SU/_rm\
SU/_sh\
SU/_stressfs\
SU/_ustests\
SU/_grind\
SU/_wc\
SU/_zombie\
SU/_pingpong\
SU/_testhello\
SU/_testhelloyou\

ifeq $(LAB),$(filter $(LAB), lock))
UPROGS += \
SU/_stats
```

```
riscv64-linux-gnu-objdump -S user/_testh
riscv64-linux-gnu-objdump -t user/_testh
mkfs/mkfs fs.img README user/xargstest.s
sertests user/_grind user/_wc user/_zomb
nmeta 46 (boot, super, log blocks 30 ino
ballocc: first 849 blocks have been alloc
ballocc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios
ce virtio-blk-device,drive=x0,bus=virtio

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ testhelloyou
Hello Rawan
$
```

```
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 pipes.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void run_command(char *cmd1[], char *cmd2[])
{
    int fd[2];
    pipe(fd);
    int rc1 = fork();
    if (rc1 == 0)
    {
        close(fd[0]);
        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);
        execvp(cmd1[0], cmd1);
        perror("execvp failed");
        exit(1);
    }
    int rc2 = fork();
    if (rc2 == 0)
    {
        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execvp(cmd2[0], cmd2);
    }
}

$ QEMU: terminated
rawan@ubuntu:~/xv6-labs-2023$ nano pipes.c
rawan@ubuntu:~/xv6-labs-2023$ gcc pipes.c -o pipes
rawan@ubuntu:~/xv6-labs-2023$ ./pipes
rawan@ubuntu:~/xv6-labs-2023$ ls
conf  fs.img  grade-lab-util  gradelib.py  kernel  LICENSE  Makefile  mkfs  pipes  pipes.c  README  user
rawan@ubuntu:~/xv6-labs-2023$ touch test.txt
rawan@ubuntu:~/xv6-labs-2023$ ./pipes
test.txt
rawan@ubuntu:~/xv6-labs-2023$
```

Conclusion

The lab demonstrated the full process of extending xv6 with new system calls and highlighted how kernel-level features interact with user programs. The pipe implementation further reinforced concepts related to process synchronization, communication, and file descriptor management in Unix systems.